

## LAB MANUAL

23ACS20: Full Stack Development – II

III B.Tech – I Semester (CSE / IT / CSO / CSM / CAI / AI)

Experiment 1: Introduction to Modern JavaScript and DOM

---

### 1a. Link JavaScript file with the HTML page

**Aim:** To write a JavaScript program that links an external JavaScript file with an HTML page using the `<script>` tag, styled with an external CSS file.

*index.html*

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Linking JavaScript File</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="navbar">
    <h2>MyWebPage</h2>
    <nav>
      <a href="#">Home</a>
      <a href="#">About</a>
      <a href="#">Contact</a>
    </nav>
  </header>

  <main class="container">
    <h1>Linking External JavaScript File</h1>
    <p id="demo">This text will change on page load.</p>
    <button id="infoBtn">Show Info</button>
  </main>

  <footer>
    <p>&copy; 2026 FSD Lab</p>
  </footer>

  <!-- Linking external JS file -->
  <script src="script.js"></script>
</body>
</html>
```

*style.css*

```
/* style.css */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: 'Segoe UI', Arial, sans-serif;
}

body {
  background-color: #f4f6f8;
  color: #222;
}
```

```

.navbar {
  background-color: #2c3e50;
  color: white;
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 14px 30px;
}

.navbar nav a {
  color: white;
  margin-left: 18px;
  text-decoration: none;
}

.container {
  max-width: 700px;
  margin: 40px auto;
  background: white;
  padding: 30px;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.1);
  text-align: center;
}

#infoBtn {
  margin-top: 16px;
  padding: 10px 20px;
  background-color: #3498db;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}

#infoBtn:hover {
  background-color: #2980b9;
}

footer {
  text-align: center;
  padding: 14px;
  color: #777;
}

```

*script.js*

```

// script.js
document.getElementById("demo").innerHTML =
  "JavaScript file linked successfully!";
console.log("External JS file is linked and running.");

document.getElementById("infoBtn").addEventListener("click", function () {
  alert("This page's JS and CSS are linked from external files.");
});

```

**Expected Output:** A styled page with a navbar, card and footer loads; the paragraph text changes to "JavaScript file linked successfully!" and clicking the button shows an alert.

---

## 1b. Select the elements in HTML page using selectors

**Aim:** To write a JavaScript program that selects HTML elements using different DOM selector methods: getElementById, getElementsByClassName, getElementsByTagName, querySelector and querySelectorAll, and styles them with CSS.

*index.html*

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Selecting Elements</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1 id="heading">Welcome to JavaScript Selectors</h1>

  <section class="card">
    <p class="highlight">Paragraph selected by class name.</p>
    <p>Another paragraph selected by tag name.</p>

    <ul id="fruitList">
      <li>Apple</li>
      <li>Banana</li>
      <li>Mango</li>
    </ul>

    
  </section>

  <script src="script.js"></script>
</body>
</html>
```

*style.css*

```
/* style.css */
body {
  font-family: 'Segoe UI', Arial, sans-serif;
  background-color: #eef2f5;
  text-align: center;
  padding: 30px;
}

h1 {
  color: #34495e;
}

.card {
  max-width: 500px;
  margin: 20px auto;
  background: white;
  padding: 24px;
  border-radius: 10px;
  box-shadow: 0 2px 6px rgba(0,0,0,0.12);
}

.highlight {
  color: #2980b9;
  font-weight: bold;
}

ul#fruitList {
  list-style: none;
  padding: 0;
}
ul#fruitList li {
```

```

        background: #f0f0f0;
        margin: 6px 0;
        padding: 8px;
        border-radius: 5px;
    }

    .selected {
        background-color: #d4efdf !important;
    }

```

*script.js*

```

// script.js

// 1. Select element by ID
let heading = document.getElementById("heading");
console.log("By ID:", heading.textContent);

// 2. Select element(s) by class name
let byClass = document.getElementsByClassName("highlight");
console.log("By Class Name:", byClass[0].textContent);

// 3. Select element(s) by tag name
let byTag = document.getElementsByTagName("p");
console.log("By Tag Name, count:", byTag.length);

// 4. Select first matching element using querySelector
let firstPara = document.querySelector("p");
console.log("querySelector:", firstPara.textContent);

// 5. Select all matching elements using querySelectorAll
let allParas = document.querySelectorAll("p");
allParas.forEach((p, index) => {
    console.log(`querySelectorAll[${index}]:`, p.textContent);
});

// 6. Select list items and add a class using selectors
let listItems = document.querySelectorAll("#fruitList li");
listItems.forEach(item => item.classList.add("selected"));

```

**Expected Output:** A styled card with a list and image is displayed; the console logs the text content selected by each method, and the list items get highlighted green.

---

## 1c. Implement the event listeners

**Aim:** To write a JavaScript program that implements event listeners on HTML elements using `addEventListener()` for events such as click, keyup, change, mouseover and mouseout, on a styled form.

*index.html*

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Event Listeners</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="wrapper">
        <h1>Event Listener Demo</h1>

        <form id="demoForm">

```

```

        <label for="nameInput">Your Name:</label>
        <input type="text" id="nameInput" placeholder="Type your name">

        <label for="colorSelect">Favourite Colour:</label>
        <select id="colorSelect">
            <option value="">-- choose --</option>
            <option value="red">Red</option>
            <option value="green">Green</option>
            <option value="blue">Blue</option>
        </select>

        <button type="button" id="clickBtn">Click Me</button>
    </form>

    <p id="output">Output will appear here...</p>
</div>

<script src="script.js"></script>
</body>
</html>

```

style.css

```

/* style.css */
body {
    font-family: 'Segoe UI', Arial, sans-serif;
    background: linear-gradient(135deg, #74ebd5, #acb6e5);
    display: flex;
    justify-content: center;
    padding-top: 40px;
}

.wrapper {
    background: white;
    padding: 30px 40px;
    border-radius: 10px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.15);
    width: 360px;
    text-align: center;
}

label {
    display: block;
    margin-top: 12px;
    text-align: left;
    font-size: 14px;
    color: #444;
}

input, select {
    width: 100%;
    padding: 8px;
    margin-top: 4px;
    border: 1px solid #ccc;
    border-radius: 5px;
}

#clickBtn {
    margin-top: 18px;
    padding: 10px 20px;
    background-color: #16a085;
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
}

```

```
#output {
  margin-top: 16px;
  font-weight: bold;
  color: #2c3e50;
}
```

*script.js*

```
// script.js

const button = document.getElementById("clickBtn");
const input = document.getElementById("nameInput");
const select = document.getElementById("colorSelect");
const output = document.getElementById("output");

// Event listener for click event
button.addEventListener("click", function () {
  output.textContent = "Button was clicked!";
});

// Event listener for input (keyup) event
input.addEventListener("keyup", function (event) {
  output.textContent = "You typed: " + event.target.value;
});

// Event listener for change event on dropdown
select.addEventListener("change", function (event) {
  output.textContent = "Favourite colour: " + event.target.value;
  output.style.color = event.target.value || "#2c3e50";
});

// Event listener for mouseover event
button.addEventListener("mouseover", function () {
  button.style.backgroundColor = "#1abc9c";
});

// Event listener for mouseout event
button.addEventListener("mouseout", function () {
  button.style.backgroundColor = "#16a085";
});
```

**Expected Output:** A styled form card appears; clicking the button, typing a name, or choosing a colour updates the output text, and the button changes colour on hover.

---

## 1d. Handle the click events for the HTML button elements

**Aim:** To write a JavaScript program that handles click events for multiple HTML button elements to build a styled increment / decrement / reset counter with a status message.

*index.html*

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Button Click Events</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="counter-card">
    <h2>Click Counter</h2>
    <p class="count-display">Count: <span id="count">0</span></p>

    <div class="btn-group">
```

```

        <button id="incrementBtn" class="btn inc">+ Increment</button>
        <button id="decrementBtn" class="btn dec">&minus; Decrement</button>
        <button id="resetBtn" class="btn reset">Reset</button>
    </div>

    <p id="status">Status: Ready</p>
</div>

<script src="script.js"></script>
</body>
</html>

```

style.css

```

/* style.css */
body {
    font-family: 'Segoe UI', Arial, sans-serif;
    background-color: #1e272e;
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    margin: 0;
}

.counter-card {
    background: #ffffff;
    padding: 30px 40px;
    border-radius: 12px;
    text-align: center;
    box-shadow: 0 6px 16px rgba(0,0,0,0.3);
    width: 320px;
}

.count-display {
    font-size: 22px;
    font-weight: bold;
    color: #2c3e50;
}

.btn-group {
    display: flex;
    justify-content: space-between;
    gap: 8px;
    margin-top: 16px;
}

.btn {
    flex: 1;
    padding: 10px;
    border: none;
    border-radius: 6px;
    color: white;
    cursor: pointer;
    font-size: 13px;
}

.btn.inc { background-color: #27ae60; }
.btn.dec { background-color: #c0392b; }
.btn.reset { background-color: #7f8c8d; }

.btn:hover { opacity: 0.85; }

#status {
    margin-top: 14px;
    font-size: 13px;
}

```

```

        color: #888;
    }
}
script.js
// script.js

let count = 0;
const countDisplay = document.getElementById("count");
const status = document.getElementById("status");

const incrementBtn = document.getElementById("incrementBtn");
const decrementBtn = document.getElementById("decrementBtn");
const resetBtn = document.getElementById("resetBtn");

// Handle click event - increment
incrementBtn.addEventListener("click", function () {
    count++;
    countDisplay.textContent = count;
    status.textContent = "Status: Incremented";
});

// Handle click event - decrement
decrementBtn.addEventListener("click", function () {
    count--;
    countDisplay.textContent = count;
    status.textContent = "Status: Decrement";
});

// Handle click event - reset
resetBtn.addEventListener("click", function () {
    count = 0;
    countDisplay.textContent = count;
    status.textContent = "Status: Reset";
});

```

**Expected Output:** A styled counter card is shown; the count updates and the status message changes on clicking Increment, Decrement, or Reset.

---

## 1e. Write a JavaScript program with three types of functions (Function Declaration, Function Definition, Arrow Function)

**Aim:** To write a JavaScript program that demonstrates the three ways of defining functions in JavaScript — Function Declaration, Function Expression (Definition), and Arrow Function — and displays the results in a styled table.

```

index.html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Types of Functions</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>Three Types of Functions in JavaScript</h1>

    <table id="resultTable">
        <thead>
            <tr><th>Function Type</th><th>Output</th></tr>
        </thead>

```

```

        <tbody>
            <!-- rows will be filled in by JavaScript -->
        </tbody>
    </table>

    <script src="script.js"></script>
</body>
</html>

```

*style.css*

```

/* style.css */
body {
    font-family: 'Segoe UI', Arial, sans-serif;
    background-color: #f9f9f9;
    padding: 40px;
    text-align: center;
}

h1 {
    color: #34495e;
    margin-bottom: 20px;
}

table#resultTable {
    margin: 0 auto;
    border-collapse: collapse;
    width: 60%;
    box-shadow: 0 2px 6px rgba(0,0,0,0.1);
}

table#resultTable th, table#resultTable td {
    border: 1px solid #ddd;
    padding: 10px 14px;
    text-align: left;
}

table#resultTable th {
    background-color: #34495e;
    color: white;
}

table#resultTable tr:nth-child(even) {
    background-color: #f2f2f2;
}

```

*script.js*

```

// script.js
// Demonstrating three types of functions in JavaScript

// i. Function Declaration
function greetDeclaration(name) {
    return "Hello (Declaration), " + name + "!";
}

// ii. Function Expression (Definition)
const greetExpression = function (name) {
    return "Hello (Expression), " + name + "!";
};

// iii. Arrow Function
const greetArrow = (name) => {
    return "Hello (Arrow), " + name + "!";
};

// Shorthand arrow function

```

```
const greetArrowShort = name => `Hello (Arrow Shorthand), ${name}!`;

const results = [
  ["Function Declaration", greetDeclaration("Mounika")],
  ["Function Expression", greetExpression("Mounika")],
  ["Arrow Function", greetArrow("Mounika")],
  ["Arrow Function (Shorthand)", greetArrowShort("Mounika")]
];

const tbody = document.querySelector("#resultTable tbody");
results.forEach([type, message] => {
  const row = document.createElement("tr");
  row.innerHTML = `<td>${type}</td><td>${message}</td>`;
  tbody.appendChild(row);
  console.log(type + ":", message);
});
```

**Expected Output:** A styled table is populated with one row per function type, each showing its greeting output for "Mounika"; the same values are also logged to the console.

---